

Exam Answer

1. **B** — Overriding lets a subclass replace/specialize a method it inherits from its superclass; this is runtime polymorphism, not encapsulation, overloading, or abstraction.
2. **B** — Encapsulation hides an object's internal state (fields) and exposes controlled access only through methods (getters/setters), protecting data integrity.
3. **B** — The 'final' keyword on a class prevents any other class from extending it.
4. **B** — Overloading means defining multiple methods with the same name but different parameter lists within the same class; return type alone doesn't distinguish overloads.
5. **C** — 'this' is a reference to the current object instance on which the instance method is being invoked.
6. **B** — Abstract classes can mix abstract (no body) and concrete (implemented) methods; they can have constructors and fields but cannot be instantiated directly.
7. **B** — Java supports single class inheritance ('extends' one class) but multiple interface implementation ('implements' many interfaces), letting a class combine several behavioral contracts.
8. **C** — 'private' restricts visibility to the declaring class only; not even subclasses or other classes in the same package can access it.
9. **B** — Polymorphism allows a single reference type (interface or superclass) to point to and interact with different concrete implementations, enabling flexible, extensible code.
10. **B** — Method calls on an object are dispatched based on its actual runtime type (Dog), not the compile-time reference type (Animal) — this is dynamic method dispatch.
11. **B** — A concrete subclass must implement all inherited abstract methods; if it doesn't, the subclass itself must be declared abstract, or compilation fails.
12. **C** — The four OOP pillars are Encapsulation, Inheritance, Polymorphism, and Abstraction; 'Compilation' is not an OOP concept at all.
13. **A** — Composition builds complex objects by embedding instances of other classes as fields, representing a 'has-a' relationship (as opposed to inheritance's 'is-a').
14. **B** — 'extends' is used for class inheritance in Java, while 'implements' is used for interfaces.
15. **B** — A constructor's job is to initialize a newly created object's state, typically setting initial field values.
16. **B** — A concrete class implementing an interface must provide implementations for all its abstract methods, unless the class itself is declared abstract.
17. **C** — Throwable is the root class of the exception hierarchy; both Exception and Error extend it.
18. **C** — IOException is a checked exception (must be declared or caught); the others (NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException) are all unchecked RuntimeExceptions.

- 19. B** — If a method doesn't handle a checked exception itself, it must declare it in a 'throws' clause so callers are aware and required to handle it.
- 20. C** — The 'finally' block runs after try/catch regardless of whether an exception occurred (except on abrupt JVM termination), typically used for cleanup.
- 21. A** — The correct syntax to manually raise an exception is 'throw new SomeException(...)', using the 'throw' keyword with a 'new' instance.
- 22. C** — NullPointerException extends RuntimeException, making it unchecked; SQLException, FileNotFoundException, and InterruptedException are all checked exceptions.
- 23. A** — Exception chaining wraps a lower-level exception inside a new one (via the cause constructor) so the original root cause is preserved while presenting a more relevant exception type.
- 24. B** — Resources used in try-with-resources must implement AutoCloseable (or its subinterface Closeable) so their close() method can be called automatically.
- 25. B** — A return (or exception) inside a finally block overrides/suppresses any return value or exception from the try/catch block — this is generally considered bad practice.
- 26. A** — Multi-catch requires the listed exception types not be related by subclassing to one another (no type may be a subclass of another in the same multi-catch clause).
- 27. B** — Custom exceptions are typically created by extending Exception (for checked) or RuntimeException (for unchecked).
- 28. B** — Exception propagation is when an exception isn't caught in the current method and moves up the call stack to be handled (or not) by a calling method.
- 29. C** — Catching a broad type like Exception or Throwable and silently swallowing it (empty catch block) hides bugs and is a well-known anti-pattern to avoid.
- 30. B** — An exception thrown inside a catch block propagates up the call stack like any other exception, unless caught by an enclosing try/catch; the finally block still runs first.
- 31. B** — Error (like OutOfMemoryError) represents serious JVM-level problems that applications generally shouldn't try to catch or recover from, unlike Exception.
- 32. C** — 'A' prints, then the RuntimeException is thrown and caught printing 'B', then finally always runs printing 'C' — output is 'ABC'.
- 33. C** — Map does not extend the Collection interface (it stores key-value pairs); List, Set, and Queue all extend Collection.
- 34. B** — ArrayList is backed by a dynamic array, giving O(1) constant-time random access by index, unlike LinkedList which requires traversal.
- 35. C** — TreeSet stores elements in a sorted tree structure and iterates them in their natural (or comparator-defined) sorted order.
- 36. B** — HashSet uses hashing internally and makes no guarantee about iteration order (order can even change over time); it also disallows duplicates.

- 37. C** — `LinkedHashMap` maintains a doubly-linked list alongside the hash table, preserving insertion order of entries during iteration.
- 38. B** — `List.of(...)` creates an immutable list; any attempt to modify it (add, remove, set) throws `UnsupportedOperationException`.
- 39. B** — `LinkedList` avoids shifting elements for middle insertions/removals (just relinks nodes), while `ArrayList` must shift subsequent elements, making `LinkedList` generally faster for that specific case.
- 40. B** — Modifying a collection structurally while iterating with a standard iterator (not using `iterator.remove()`) throws `ConcurrentModificationException` via the fail-fast mechanism.
- 41. C** — `Map` is the interface that provides key-value pair storage in the Java Collections Framework.
- 42. B** — `Collections.unmodifiableList` wraps a list in a read-only view; any structural modification attempt through that view throws `UnsupportedOperationException` (the underlying list can still change if accessed directly).
- 43. B** — `Queue` represents a FIFO (first-in-first-out) structure, ideal for processing items in the order they arrive.
- 44. C** — A well-implemented `HashMap` offers average $O(1)$ constant-time lookup by key due to hashing, assuming a good hash distribution.
- 45. B** — `map.entrySet()` returns a `Set` of key-value pair entries, allowing iteration over both keys and values together.
- 46. B** — Putting a key that already exists in a `HashMap` replaces the old value associated with that key with the new one (and returns the old value).
- 47. B** — `Set` is defined specifically to disallow duplicate elements; `List` (including `ArrayList`) permits duplicates, and `Queue` also generally allows them.
- 48. B** — Generics provide compile-time type safety, catching type errors early and eliminating the need for explicit casting when retrieving elements.
- 49. B** — The contract requires that if two objects are equal per `equals()`, they must produce the same `hashCode()`; the reverse (same hash implies equal) is not required.
- 50. B** — `Object`'s default `equals()` simply compares references (memory addresses), behaving identically to the `==` operator, unless overridden.
- 51. C** — Overriding `hashCode()` alongside `equals()` ensures consistent, correct behavior in hash-based collections like `HashMap`/`HashSet`, which rely on `hashCode()` to locate buckets before confirming with `equals()`.
- 52. B** — Unequal objects are not required to have different `hashCode()` values — hash collisions between unequal objects are allowed and expected to some degree.
- 53. B** — Without a matching `hashCode()`, equal objects can hash into different buckets, causing `HashMap`/`HashSet` to fail to find or deduplicate them correctly.
- 54. A** — The `equals()` contract requires it to be reflexive (`x.equals(x)`), symmetric (`x.equals(y) == y.equals(x)`), transitive, and consistent across repeated calls.

- 55. B** — By contract, `x.equals(null)` must return false for any non-null object `x`, not throw an exception.
- 56. B** — Typical generated `equals()` methods first check reference equality (`this == obj`, a shortcut) and null/class-type checks, then compare individual field values.
- 57. B** — If a field used in `hashCode()/equals()` changes after the object is inserted as a `HashMap` key, its hash bucket location becomes inconsistent, making the entry effectively unfindable.
- 58. B** — `java.util.Objects` provides null-safe helper methods like `equals(a, b)` and the varargs `hash(a, b, c, ...)` for building `hashCode()`.
- 59. B** — `Objects.equals(a, b)` safely handles null by checking for null before comparing, avoiding a `NullPointerException` that `a.equals(b)` would throw if `'a'` were null.
- 60. B** — Without overriding, a class inherits `Object`'s identity-based `equals()/hashCode()`, so `HashMap` keys are compared by reference, not by logical content.